2520090063
R.Radhakrishna

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
Radhakrishna@04:~# nano skillweek04.c
Radhakrishna@04:~# gcc skillweek04.c -o skillweek04
Radhakrishna@04:~# ./skillweek04
```

Code:

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX_INPUT 1024
#define MAX_TOKENS 100
int main() {
   char input[MAX_INPUT];
   char *tokens[MAX_TOKENS];
   int count = 0;
   printf("myShell> ");
   if (fgets(input, sizeof(input), stdin) == NULL) {
      return 0
   }
   input[strcspn(input, "\n")] = '\0';
   if (strlen(input) == 0) {
      printf("Empty command.\n");
```

```
    return 0;
  }
  char *token = strtok(input, " \t");
  while (token != NULL && count < MAX_TOKENS) {
    tokens[count++] = token;
    token = strtok(NULL, " \t");
  }
  printf("\n--- Parsing Result ---\n");
  for (int i = 0; i < count; i++) {
    printf("Argument %d: %s\n", i + 1, tokens[i]);
  }
  printf("\nTotal tokens: %d\n", count);
  return 0;
}
```

Output:

```
Radhakrishna@04:~# nano skillweek04.c
Radhakrishna@04:~# gcc skillweek04.c -o skillweek04
Radhakrishna@04:~# ./skillweek04
myshell$ echo Hello
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Parent Process PID: 511
Created Child PID: 512
Child Process
PID = 512
Hello
Child exited with status: 0
myshell$ echo 'Hello World'
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
Radhakrishna@04:~# nano skillweek04.c
Radhakrishna@04:~# gcc skillweek04.c -o skillweek04
Radhakrishna@04:~# ./skillweek04
myshell$ echo Hello
```

code:

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_INPUT 1024
#define MAX_TOKENS 100
int main() {
   char input[MAX_INPUT];
   char *tokens[MAX_TOKENS];
   int count = 0;
   printf("myShell> ");
   if (fgets(input, sizeof(input), stdin) == NULL) {
      return 0;
   }
   input[strcspn(input, "\n")] = '\0';
   if (strlen(input) == 0) {
      printf("Empty command.\n");
      return 0;
   }
```

```
int in_single_quote = 0;
char buffer[MAX_INPUT];
int j = 0;
for (int i = 0; input[i] != '\0'; i++) {
    if (input[i] == '\'') {
        in_single_quote = !in_single_quote;
    }
    else if (input[i] == ' ' || input[i] == '\t') {
        if (in_single_quote) {
            buffer[j++] = input[i];
        }
        else {
            if (j > 0) {
                buffer[j] = '\0';
                tokens[count] = malloc(strlen(buffer) + 1);
                strcpy(tokens[count], buffer);
                count++;
                j = 0;
            }
        }
    }
    else {
        buffer[j++] = input[i];
    }
}
if (in_singe_quote) {
```

```
        printf("Syntax Error: Unmatched single quote.\n");
        for (int i = 0; i < count; i++)
            free(tokens[i]);
        return 1;
    }
    if (j > 0) {
        buffer[j] = '\0';
        tokens[count] = malloc(strlen(buffer) + 1);
        strcpy(tokens[count], buffer);
        count++;
    }
    printf("\n--- Parsing Result ---\n");
    for (int i = 0; i < count; i++) {
        printf("Argument %d: %s\n", i + 1, tokens[i]);
    }
    for (int i = 0; i < count; i++)
        free(tkens[i]);
    return 0;
}
```

Output:

```
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Parent Process PID: 511
Created Child PID: 515
Child Process
PID = 515
Hello World
Child exited with status: 0
myshell$ echo "Hello World"
```

Task-3:

```
Radhakrishna@04:~# nano skillweek04.c
Radhakrishna@04:~# gcc skillweek04.c -o skillweek04
Radhakrishna@04:~# ./skillweek04
myshell$ echo Hello
```

Code:

```
#include  <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>

#define MAX_INPUT 1024

#define MAX_TOKENS 100

void expand_variables(char *input, char *output) {

   int i = 0, j = 0;

   while (input[i] != '\0') {

      if (input[i] == '$') {

      i++;

         char variable[100];

         int k = 0;

         while (isalnum(input[i]) || input[i] == '_') {
```

```
            variable[k++] = input[i++];
        }
        variable[k] = '\0';
        char *value = getenv(variable);
        if (value != NULL) {
            strcpy(&output[j], value);
            j += strlen(value);
        }
    }
    else {
        output[j++] = input[i++];
    }
  }
  output[j] = '\0';
}
int main() {
  char input[MAX_INPUT];
  char *tokens[MAX_TOKENS];
  int count = 0;
  printf("myShell> ");
  if (fgets(input, sizeof(input), stdin) == NULL)
    return 0;
  input[strcspn(input, "\n")] = '\0';
  if (strlen(input) == 0) {
    printf("Empty command.\n");
    return 0;
```

```
}


int in_double_quote = 0;
char buffer[MAX_INPUT];
int j = 0;
for (int i = 0; input[i] != '\0'; i++) {
    if (input[i] == '"') {
        in_double_quote = !in_double_quote;
    }
    else if ((input[i] == ' ' || input[i] == '\t') &&
             !in_double_quote) {
        if (j > 0) {
            buffer[j] = '\0';
            tokens[count] = malloc(MAX_INPUT);
            char expanded[MAX_INPUT];
            expand_variables(buffer, expanded);
            strcpy(tokens[count], expanded);
            count++;
            j = 0;
        }
    }
    else {
        buffer[j++] = input[i];
    }
}
```

```
    if (in_double_quote) {
        printf("Syntax Error: Unmatched double quote.\n");
        for (int i = 0; i < count; i++)
            free(tokens[i]);
        return 1;
    }
    if (j > 0) {
        buffer[j] = '\0';
        tokens[count] = malloc(MAX_INPUT);
        char expanded[MAX_INPUT];
        expand_variables(buffer, expanded);
        strcpy(tokens[count], expanded);
        count++;
    }
    printf("\n--- Parsed Arguments ---\n");
    for (int i = 0; i < count; i++) {
        printf("Argument %d: %s\n", i + 1, tokens[i]);
    }
    for (int i = 0; i < count; i++)
        free(tokens[i]);
    return 0;
}
```

Output:

```
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Parent Process PID: 511
Created Child PID: 516
Child Process
PID = 516
Hello World
Child exited with status: 0
myshell$ echo Hello\ World
```

Task-4:

```
Radhakrishna@04:~# nano skillweek04.c
Radhakrishna@04:~# gcc skillweek04.c -o skillweek04
Radhakrishna@04:~# ./skillweek04
myshell$ echo Hello
```

Code:

```
 GNU nano 7.2                                                                skil
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int main() {

    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }

    char buffer[MAX_INPUT];
    int j = 0;
    int escaped = 0;

    for (int i = 0; input[i] != '\0'; i++) {

        if (escaped) {
            buffer[j++] = input[i];
            escaped = 0;
        }
        else if (input[i] == '\\') {
            escaped = 1;
        }
        else if (input[i] == ' ' || input[i] == '\t') {

            if (j > 0) {
                buffer[j] = '\0';

                tokens[count] = malloc(strlen(buffer) + 1);
                strcpy(tokens[count], buffer);

                count++;
                j = 0;
            }
        }
        else {
            buffer[j++] = input[i];
        }
    }
```

```
    }

    if (escaped) {
        printf("Syntax Error: Escape character at end of input.\n");
        return 1;
    }

    if (j > 0) {
        buffer[j] = '\0';

        tokens[count] = malloc(strlen(buffer) + 1);
        strcpy(tokens[count], buffer);

        count++;
    }

    printf("\n--- Parsed Arguments ---\n");

    for (int i = 0; i < count; i++) {
        printf("Argument %d: %s\n", i + 1, tokens[i]);
    }

    for (int i = 0; i < count; i++)
        free(tokens[i]);

    return 0;
}
```

Output:

```
myshell$ echo Hello\ World
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Parent Process PID: 511
Created Child PID: 517
Child Process
PID = 517
Hello World
Child exited with status: 0
myshell$ ls
```

Task-5:

```
Radhakrishna@04:~# nano skillweek04.c
Radhakrishna@04:~# gcc skillweek04.c -o skillweek04
Radhakrishna@04:~# ./skillweek04
myshell$ echo Hello
```

Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100
int main() {
    char input[MAX_INPUT];
    printf("myshell$ ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;
    input[strcspn(input, "\n")] = '\0';
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }
    char *args[MAX_ARGS];
    int count = 0;
    char *token = strtok(input, " \t");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }
    args[count] = NULL;
    pid_t parent_pid = getpid();
    printf("Parent Process PID: %d\n", parent_pid);
```

```
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
]
    if (pid == 0) {
        // Child process
        printf("Created Child PID: %d\n", getpid());
        printf("Child Process PID: %d\n", getpid());
        execvp(args[0], args);
        // If execvp returns, execution failed
        perror(args[0]);
        exit(2);
    }
    else {
        // Parent process
        int status;
        waitpid(pid, &status, 0);
        if (WIFEXITED(status)) {
            printf("Child exited with status: %d\n",
                   WEXITSTATUS(status));
        }
        else {
            printf("Child terminated abnormally.\n");
```

```
    }

  }

  return 0;

}
```

Output: